PLATFORM DOCUMENTATION

StatSkillAI

AI-Powered Competency Measurement, Skill-Gap Diagnostic & Career Development Platform

📋 Aligned with National Education Policy (NEP) 2020 & Mission Karmayogi

VERSION	GENERATED	CLASSIFICATION
Production v1.0	October 2026	Technical & Product

SECTION 01

Platform Overview

StatSkillAI is a full-stack, production-ready web application that bridges the gap between **competency assessment** and **career development** for India's workforce. Built on real-time AI capabilities (Google Gemini), it serves three distinct user cohorts aligned with the vision of NEP 2020 and Mission Karmayogi.

3	4	16	10
User Roles	Competency Domains	Database Models	API Route Modules

Who Uses StatSkillAI?

User Type	Profile	Primary Goal
🎓 LEARNER	Students, Government Employees, Private-Sector Professionals	Assess competencies, close skill gaps, earn opportunities
👤 MENTOR	Verified domain experts (ISI, NIC, MOSPI alumni)	Guide mentees via directives, course recommendations, direct messaging
🔑 ADMIN	Institutional administrators (NSDC, MoSPI)	Monitor workforce analytics, manage course catalogs, track applicants

The Problem StatSkillAI Solves

✕ PROBLEM 1 — NO OBJECTIVE SKILL MEASUREMENT

Students and employees lack tailored, objective tools to diagnose specific competency deficits across technical, statistical, digital, and behavioural domains.

✕ PROBLEM 2 — DISCONNECTED WORKFORCE DATA

Government ministries cannot monitor, benchmark, or close critical skill gaps across their workforce. Analytics are static and disconnected from actual learning activity.

✕ PROBLEM 3 — FRAGMENTED COURSE CATALOGS

SWAYAM, NPTEL, iGOT Karmayogi, and NCS are siloed. Learners cannot receive personalised, sequenced recommendations from verified national portals.

✕ PROBLEM 4 — STATIC ASSESSMENTS

Existing assessments do not adapt to individual capability in real time. Questions repeat, difficulty does not scale, and results do not feed back into career matching.

The StatSkillAI Solution

- ✓ Psychometrically sound, AI-adaptive competency evaluations across 4 core domains
- ✓ Automated skill-gap synthesis with severity classification (High / Medium / Low)
- ✓ Curated, personalised learning pathways from verified Indian national portals
- ✓ Real-time 1-on-1 mentorship via Socket.io with typing indicators and message threads
- ✓ Institutional workforce heatmaps with departmental drill-down and AI policy advisories
- ✓ AI-generated document quiz MCQs from uploaded PDFs or pasted text
- ✓ Bilingual interface (English / Hindi) with WCAG 2.1 AA accessibility compliance

SECTION 02

Technical Architecture

Technology Stack

Layer	Technology	Purpose
Frontend Framework	React 19 + TypeScript + Vite 8	Fast SPA with full type-safety and hot module replacement
UI & Styling	Tailwind CSS 4	Responsive, accessible design with high-contrast mode support
Charting	Recharts 3	Radar charts, line charts, bar charts, department heatmaps
Icons	Lucide React	Consistent icon system across all components
Animations	Motion (Framer Motion v12)	Smooth page transitions and micro-interactions
Backend Runtime	Node.js 25 + TypeScript (tsx)	ESM TypeScript execution without build compilation step
Web Framework	Express 4.21	REST API, middleware composition, static file serving
Real-Time	Socket.io 4.8	JWT-authenticated WebSocket channels for live messaging
Database ORM	Prisma 6.19 + PostgreSQL	Type-safe database access, schema migrations, connection pooling
AI Integration	@google/genai v2.4 (Gemini)	Question generation, gap analysis, learning paths, AI chat
Authentication	JWT (jsonwebtoken) + bcryptjs	Dual-token sessions (7-day access + 30-day refresh)
Security	Helmet + express-rate-limit	Security headers, DDoS protection, brute-force mitigation
Routing (Client)	React Router v7	Hash-based SPA routing with role-based route guards

System Architecture Diagram




BROWSER (SPA Client)
React 19 + Tailwind CSS 4 + Recharts + Socket.io-client
Learner Pages
Mentor Pages
Admin Pages
Auth Pages (Login/Reg)
ApiService (JWT Bearer + REST)
Socket.io Client (JWT auth token)
HTTPS / WSS
EXPRESS SERVER (server.ts)
Helmet · CORS · Rate Limiter · Cookie Parser · Body JSON
JWT Middleware (authenticate)
requireRole · requireOwnership (RBAC)
/api/auth /api/profile /api/assessments /api/gaps
/api/paths /api/courses /api/opportunities /api/mentor
/api/analytics /api/quizzes /api/gemini/*
Socket.io 4.8 (WebSocket)
Gemini AI Proxy (multi-model fallback)
PostgreSQL (Prisma) 16 Tables
Google Gemini API (gemini-3.5-flash etc.)

Database Schema (16 Models)

Model	Key Fields	Purpose
`User`	id, name, email, passwordHash, role, profileType	Base identity for all roles
`LearnerProfile`	userId, skills[], interests[], targetDomainScores, careerGoal	Learner biographical & competency targets
`AssessmentAttempt`	userId, domain, score, questions[], userAnswers[], earnedWeight	Full per-attempt assessment record
`QuestionRecord`	domain, difficulty, question, options[], correctIndex, explanation	Seeded MCQ bank for offline fallback
`GapAnalysis`	userId, gaps[], strengths[], domainScores{}, priorityOrder[]	AI-synthesized competency diagnostic
`LearningPath`	userId, title, careerGoal, progressPercent	Sequenced skill progression container
`LearningStep`	learningPathId, title, domain, type, status, isMentorRecommended	Individual step within learning path
`Course`	title, domain, provider, source, duration, level, portalUrl	National course catalog (SWAYAM/NPTEL)
`Opportunity`	title, organisation, sector, type, requiredSkills[], minDomainLevels{}	Jobs, fellowships, internships
`Application`	opportunityId, userId, status, matchPercentage	Candidate applications with status pipeline
`Mentor`	userId, name, field, isVerified, rating, languages[]	Verified domain expert profiles
`Mentorship`	mentorId, learnerId, status, feedbackNotes[], goalMessage	Active mentorship relationships
`DirectMessage`	mentorshipId, senderId, text, timestamp	Real-time 1-on-1 message threads
`QuizRecord`	userId, questions[], userAnswers[], score, sourceType	AI document-generated quiz history
`Organisation`	name, sector, description	Nodal government entities

Model	Key Fields	Purpose
`AppSettings`	userId, language, highContrast, fontSize	Per-user accessibility & locale preferences

SECTION 03

Learner Features

Feature 1 — AI-Adaptive Competency Assessment

A 5-question adaptive psychometric assessment for each of the 4 domains. Difficulty dynamically adjusts based on real-time answer accuracy using a simple Elo-like difficulty ladder.

🧠 Dynamic Difficulty Adjustment

Starts at Medium difficulty. Correct → escalates to Hard (3 pts). Wrong → drops to Easy (1 pt). Final score = earned weight ÷ max possible weight × 100.

🤖 Gemini AI Question Generation

Each question is generated uniquely via Gemini API. Prompts specify domain, difficulty, learner profile type, and language preference. Previous questions are excluded to prevent repetition.

📚 Offline Fallback Question Bank

If Gemini is unavailable, a curated bank of 60+ domain-specific MCQs (Easy/Medium/Hard per domain) activates. Questions include detailed pedagogical explanations.

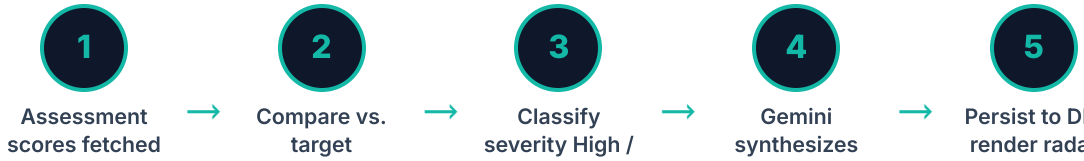
⏱ 60-Second Timer per Question

Optional 60s per-question timer auto-submits on expiry. Timer resets between questions. Togglable for accessibility compliance.

After assessment, scores are immediately persisted to the database and the **Gap Analysis is automatically recalculated server-side** — no manual refresh required.

Feature 2 — AI Skill-Gap Analysis Engine

Upon completing an assessment or clicking "Recalculate", the system computes and synthesizes a comprehensive diagnostic report.



	benchmarks	Med / Low	summary & priorities	chart

Gap Severity	Calculation	Priority
🔴 **High**	Target − Current ≥ 25 points	Addressed first in learning path
🟡 **Medium**	15 ≤ gap < 25 points	Secondary priority
🟢 **Low**	gap < 15 points	Maintenance recommendation

Feature 3 — Personalised Learning Path Generator

Gemini synthesizes a sequenced 6–10 step learning pathway from the gap analysis, mapping each step to a specific course from SWAYAM, NPTEL, iGOT Karmayogi, or NCS.

- ✓ Mentor-recommended steps are **preserved across AI regenerations** — no overwrites
- ✓ Step statuses: Not started → In progress → Completed (persisted to database)
- ✓ Domain filter to view steps by Technical / Statistical / Digital / Behavioural
- ✓ Progress percentage auto-calculated from completed steps ratio
- ✓ Completing steps triggers domain score refresh via `refreshScores()`

Feature 4 — AI Document Quiz Generator

Learners upload a PDF or paste text → Gemini generates N custom MCQs grounded strictly in the provided material.

- **Input modes:** PDF upload (max 10 MB, base64 encoded inline) or plain text paste
- **Question count:** 1–10 questions per generation
- **Difficulty levels:** Easy, Medium, Hard, or Mixed
- **Quiz history:** All completed quizzes persist to database with scores and individual answers
- **Retry incorrect:** Re-take only wrong answers from a completed quiz
- **Export:** Download quiz as .txt file for offline study

Feature 5 — Career Opportunity Matching

An intelligent matcher computes a composite match percentage for every open opportunity against the learner's current competency profile:

MATCH FORMULA

Match % = (Skill Overlap Ratio × 55%) + (Domain Level Ratio × 45%), clamped between 45% and 98%

- Filter by opportunity type: Job, Internship, Fellowship, Training Programme
- Filter by sector (Public, Private, Research) and minimum match threshold
- One-click application creates a persisted Application record in the database
- Real-time status updates dispatched to learner via Socket.io when admin changes status

Feature 6 — Human Mentorship Platform

- ✓ Directory of verified domain experts (Statistical, Digital, Behavioural specialisations)
- ✓ Filter by specialisation field and language preference (English / Hindi)
- ✓ Send mentorship request with a personalised goal message
- ✓ Real-time 1-on-1 direct messaging via Socket.io with typing indicators
- ✓ Mentor-issued written directives and course recommendations visible in learner's path
- ✓ Rate completed mentorships with star rating and comments

Feature 7 — Floating AI Learning Assistant

A persistent chat bubble (bottom-right corner) gives access to a contextual Gemini AI assistant that is aware of the learner's competency scores, active skill gaps, and career goals. Starter prompts guide first-time users. Full chat history persists across sessions.

SECTION 04

Mentor & Admin Features

Mentor Workspace

📋 Mentee Roster Dashboard

View all assigned mentees and pending mentorship requests. Accept or Decline requests with status tracking. View mentee's goal message, profile type, and request date.

🔍 Mentee Profile Drilldown

Deep-dive view of any mentee's radar competency chart, domain scores, identified skill gaps, and current learning path steps. Full read access to their diagnostic data.

📝 Written Directives

Issue written feedback notes ("directives") directly to the mentee. Notes are stored in the Mentorship record and broadcast via Socket.io as a real-time notification to the learner.

📚 Course Recommendations

Select any course from the national catalog and add it as a mentor-recommended step to the mentee's learning path. Recommendations are preserved even when learner regenerates their AI path.

💬 Direct Message Thread

Full real-time 1-on-1 chat with mentee via Socket.io. Messages persist to database. Typing indicators show when the other party is composing a message.

🔒 Row-Level Security

Server enforces that only the assigned mentor–mentee pair can access a given mentorship thread. Cross-account access returns 403 Forbidden.

Admin Control Center

📊 Workforce Analytics Dashboard

The most powerful admin view — a real-time institutional intelligence panel combining *actual registered learner data* with seeded departmental employees (65 org employees across 8 departments).

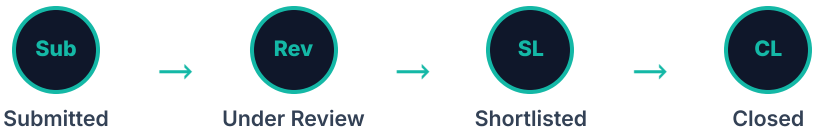
- ✓ Department × Domain skill-gap heatmap with color-coded severity cells (Critical / Moderate / Target Met / Advanced)
- ✓ Click any cell to drill down into individual employees within that department and domain
- ✓ Privacy mode toggle anonymises all employee names for DPDP Act compliance
- ✓ 4 KPI widgets: Total employees, participation rate, completion rate, workforce average score
- ✓ Bar chart: Average scores by domain across all departments
- ✓ Line chart: Training progression over time
- ✓ Gemini AI generates institutional risk advisory and actionable recommendations (with session cache)
- ✓ Export workforce data to CSV for offline reporting

📚 Course & Opportunity Catalog Manager

Action	Description
Create Course	Add new course with title, domain, provider, source (SWAYAM/NPTEL/iGOT), duration, difficulty level, portal URL, and skills covered
Delete Course	Remove outdated courses; cascading effects warn admin
Create Opportunity	Add jobs, internships, fellowships with required skills and domain threshold prerequisites
Delete Opportunity	Remove closed openings with confirmation prompt

👥 Applicant Tracking Pipeline

Full recruiter-style applicant tracker showing all learner applications across all opportunities. Admin can move any application through the status pipeline:



Each status change broadcasts a real-time Socket.io notification to the learner's browser — they receive an animated toast notification without refreshing.

🛡️ Mentor Verification Authority

Admin can grant or revoke "Verified" badge for any registered mentor. Verification records the date. Unverified mentors appear in the directory but learners are advised to prioritise verified experts.

⚙️ System Settings & Data Administration

- ✓ Toggle interface language between English and हिन्दी
- ✓ High-contrast mode (WCAG 2.1 AA compliance toggle)
- ✓ Font size controls (Small / Default / Large)
- ✓ Factory data reset — restores all prototype demo collections to initial state

SECTION 05

Security & Authentication

Authentication System

🔐 Bcrypt Password Hashing

All passwords hashed with bcryptjs using 12 salt rounds. Server enforces password complexity: min 8 chars, uppercase, lowercase, digit, and special character.

🎫 Dual-Token JWT Session

Access Token: 7-day signed JWT. Refresh Token: 30-day signed JWT. Both transmitted via HTTP-only cookies AND Bearer header for maximum compatibility.

🔄 Automatic Token Refresh

`POST /api/auth/refresh` endpoint validates the refresh token and issues a fresh access token seamlessly without forcing re-login.

🔐 Socket.io JWT Auth

All WebSocket connections require a valid JWT token passed in the handshake auth object. Unauthenticated socket connections are immediately rejected.

Role-Based Access Control (RBAC)

Route	Allowed Roles	Additional Guards
`/api/assessments`	learner, admin	userId must match token userId
`/api/gaps`	learner, admin	userId scoped to authenticated user
`/api/paths`	learner	Mentor steps preserved on regeneration
`/api/quizzes`	learner	Per-user isolation enforced
`/api/profile`	learner	PUT scoped to own userId only
`/api/courses` (write)	admin only	GET is public (no auth required)
`/api/opportunities` (write)	admin only	Apply endpoint: learner only
`/api/analytics/workforce`	admin only	Full workforce data access
`/api/mentorship` messages	Mentee or assigned Mentor	Row-level: both userId must be in mentorship record
`/api/mentorship/mentors/:id/verify`	admin only	Verification date auto-stamped

Security Middleware Stack

```
// Applied globally to every request
app.use(helmet({                // Security headers (X-Frame-Options, CSP, etc.)
  contentSecurityPolicy: false  // Disabled for Vite HMR compatibility
}));
app.use(cors({ origin: true, credentials: true }));
app.use(express.json({ limit: '25mb' })); // Allows PDF base64 payloads

// Rate Limiters
const authLimiter = rateLimit({ windowMs: 15min, max: 100 reqs });
const apiLimiter  = rateLimit({ windowMs: 1min,  max: 300 reqs });
app.use('/api/auth/login', authLimiter);    // Brute-force protection
app.use('/api/auth/register', authLimiter);
```

Password Validation Rules

Rule	Requirement
Minimum Length	≥ 8 characters
Maximum Length	≤ 128 characters
Uppercase Letter	At least one [A-Z]
Lowercase Letter	At least one [a-z]
Digit	At least one [0-9]
Special Character	At least one from !@#$%^&*()_+-= etc.

DEMO PASSWORD

All seed demo accounts use the password: `DemoPassword123!` — hashed with bcrypt 10-rounds in the seed script.

SECTION 06

API Reference

Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/register	None	Register new user; returns JWT access + refresh tokens
POST	/api/auth/login	None	Login with email + password; returns JWT tokens in cookie & body
GET	/api/auth/me	JWT	Verify session and return current user + profile
POST	/api/auth/refresh	Refresh JWT	Issue new access token using valid refresh token
POST	/api/auth/logout	JWT	Clear session cookies
POST	/api/auth/demo-switch	None	Issue real JWT for any seed demo user (demo mode only)

Core Feature Endpoints

Method	Endpoint	Role	Description
GET	/api/profile	learner	Fetch authenticated learner's profile
PUT	/api/profile	learner	Update profile (skills, interests, targets, career goal)
GET	/api/profile/domain-scores	learner	Return latest scores from most recent attempt per domain
GET	/api/assessments/attempts	learner	Fetch all completed assessment attempts (latest 20)
POST	/api/assessments/submit	learner	Submit completed assessment; triggers gap recalculation
GET	/api/gaps	learner	Fetch current gap analysis report
POST	/api/gaps/save	learner	Upsert gap analysis (after Gemini generation)
GET	/api/paths	learner	Fetch learning path with all steps (ordered)
POST	/api/paths/save	learner	Save/regenerate path; preserves mentor steps
PATCH	/api/paths/steps/:id/status	learner	Update step completion status
GET	/api/courses	public	Fetch entire course catalog
POST	/api/courses	admin	Create new course entry
DELETE	/api/courses/:id	admin	Remove course from catalog
GET	/api/opportunities	public	Fetch all verified opportunities
POST	/api/opportunities/:id/apply	learner	Submit application; emits Socket.io notification
GET	/api/analytics/workforce	admin	Full workforce roster + KPI aggregates
POST	/api/gemini/generate	JWT (any)	Proxy Gemini generateContent with model fallback

Method	Endpoint	Role	Description
`POST`	/api/gemini/chat	JWT (any)	Proxy Gemini multi-turn chat with model fallback
`GET`	/api/health	public	Health check: DB connectivity + Gemini API key status

Gemini AI Model Fallback Chain

AUTOMATIC MODEL SELECTION

The server tries models in order until one succeeds: `GEMINI_MODEL` env var → `gemini-3.5-flash` → `gemini-3-flash-preview` → `gemini-3.1-flash-lite` → `gemini-3.8-flash`. If all fail, the client activates the local offline fallback question bank.

Socket.io Event Reference

Event	Direction	Payload	Description
`join_mentorship`	Client → Server	mentorshipId: string	Join real-time room for message thread
`leave_mentorship`	Client → Server	mentorshipId: string	Leave message room on navigation
`typing_start`	Client → Server	{ mentorshipId }	Broadcast typing indicator to other participant
`typing_stop`	Client → Server	{ mentorshipId }	Remove typing indicator
`new_message`	Server → Client	DirectMessage object	Real-time new message delivery to room
`user_notification`	Server → Client	{ type, title, message }	Toast notification for application status updates

SECTION 07

Setup & Deployment

Local Development Setup

Prerequisites: Node.js 20+, Git, PostgreSQL (local or cloud e.g. Neon/Supabase)

```
# 1. Clone the repository
git clone https://github.com/yourrepo/statskillai.git
cd statskillai

# 2. Install all dependencies
npm install

# 3. Copy environment variables and fill in your keys
cp .env.example .env

# Required variables in .env:
GEMINI_API_KEY=your_google_gemini_api_key
DATABASE_URL=postgresql://user:pass@localhost:5432/statskillai
JWT_SECRET=your_secure_random_secret_min_32_chars
PORT=3000

# 4. Push Prisma schema to database
npx prisma db push

# 5. Seed demo users, courses, and data
npx tsx prisma/seed.ts

# 6. Generate Prisma Client (auto-done on npm install)
npx prisma generate

# 7. Start the development server (backend + Vite frontend)
npm run dev   # → http://localhost:3000
```

Environment Variables

Variable	Required	Description
`GEMINI_API_KEY`	✅ Required	Google Gemini API key for all AI features
`DATABASE_URL`	✅ Required	PostgreSQL connection string (supports Neon, Supabase, local)
`JWT_SECRET`	✅ Required	Secret for signing access tokens (min 32 chars)
`JWT_REFRESH_SECRET`	Optional	Separate secret for refresh tokens (defaults to JWT_SECRET)
`PORT`	Optional	HTTP listener port (default: 3000)
`NODE_ENV`	Optional	Set to "production" to serve static dist/ folder instead of Vite
`GEMINI_MODEL`	Optional	Override default AI model (falls back to gemini-3.5-flash)
`VITE_API_BASE_URL`	Production only	Points frontend to deployed backend URL (e.g. Render.com URL)

NPM Scripts

Script	Command	Description
`npm run dev`	tsx server.ts	Start server with Vite middleware for HMR development
`npm run build`	vite build	Compile frontend to production-optimised dist/ bundle
`npm run start`	prisma db push && tsx prisma/seed.ts && tsx server.ts	Production start: migrate + seed + run server
`npm run lint`	tsc --noEmit	TypeScript type-checking without emitting files
`npm run prisma:generate`	prisma generate	Regenerate Prisma Client after schema changes
`npm run prisma:push`	prisma db push	Apply schema changes to database (dev mode)
`npm run prisma:seed`	tsx prisma/seed.ts	Run database seeder (idempotent upserts)

Production Deployment

🌐 Frontend — Vercel

Connect Git repository. Set Build Command: `npm run build`. Set Output Directory: `dist`. Add `VITE_API_BASE_URL` environment variable pointing to your Render backend URL.

🖥️ Backend — Render

Create a Web Service, connect Git. Set Start Command: `npm start`. Add `DATABASE_URL`, `GEMINI_API_KEY`, `JWT_SECRET`, `PORT=8080`, `NODE_ENV=production`.

🗄️ Database — Neon / Supabase

Create a free PostgreSQL project on Neon (neon.tech) or Supabase (supabase.com). Copy the connection string to `DATABASE_URL`. Prisma handles all migrations automatically.

🔑 Get Gemini API Key

Visit **aistudio.google.com** → Sign in → Create API Key. Free tier supports thousands of requests/day. Add key to `GEMINI_API_KEY` in both Vercel and Render.

Demo Accounts (Post-Seed)

Role	Name	Email	Password
LEARNER	Aarav Sharma (Student)	aarav.sharma@univ.edu.in	DemoPassword123!
LEARNER	Priya Verma (Govt. Employee)	priya.verma@mospi.gov.in	DemoPassword123!
LEARNER	Vikram Malhotra (Private)	vikram.m@analyticscloud.in	DemoPassword123!
MENTOR	Dr. Anita Rao	dr.anita.rao@isical.ac.in	DemoPassword123!
MENTOR	Rajesh Kulkarni	rajesh.kulkarni@nic.in	DemoPassword123!
ADMIN	Sunil Mehta	sunil.mehta@nsdcindia.org	DemoPassword123!

SECTION 08

Future Features & Implementation Roadmap

The following enhancements are recommended to significantly improve user experience, engagement, and institutional value. Each feature includes an implementation guide for the current codebase.

🚀 High-Impact: Implement First

1

High Priority

Email Notification System (Nodemailer + SendGrid)

What: Send email alerts when application status changes, mentorship is accepted/declined, or mentor posts a directive. Currently only Socket.io real-time notifications exist — users who are offline miss updates.

How to implement: Install `nodemailer` + `@sendgrid/mail`. Create `server/services/emailService.ts`. Trigger from `opportunities.ts` (status change) and `mentorship.ts` (request accepted). Add `SENDGRID_API_KEY` to .env.

2

High Priority

Verifiable Digital Credentials / Certificates (PDF export)

What: Auto-generate a downloadable competency certificate (PDF) when a learner completes their learning path or scores above 75% in any domain. Certificates include a UUID-based verification URL.

How to implement: Use `puppeteer` or `pdf-lib` on the server. Create `GET /api/certificates/:userId` endpoint. Add a "Download Certificate" button to `LearnerDashboard.tsx`. Store certificate hash in a new `Certificate` Prisma model.

3

High Priority

Video Call Integration (WebRTC / Whereby Embedded)

What: Enable mentor and mentee to initiate a scheduled or instant video call directly within the mentorship thread. Add meeting notes post-call.

How to implement: Integrate Whereby Embedded SDK (or 100ms.live free tier) — create a meeting room via their API and render the meeting URL in an iframe within `MentorshipPage.tsx`. Store meeting links in the `Mentorship` model's `meetingSchedule` field.

4

High Priority

Server-Side PDF Text Extraction (pdf-parse)

What: Currently the Quiz Generator sends raw base64 PDF to Gemini (expensive, slow, large payloads). Extract text server-side first, then pass clean text to Gemini.
How to implement: Install `pdf-parse` npm package. Create `POST /api/quizzes/parse-pdf` endpoint that accepts a multipart file upload. Return extracted text to client. Client then calls the existing quiz generation flow with text instead of base64.

Medium Priority Enhancements

5

Medium Priority

Gamification — Badges, Streaks & Leaderboard

What: Award digital badges (e.g. "Statistical Expert", "7-Day Streak", "Mentor's Pick") based on assessment performance and consistency. Add an opt-in institutional leaderboard for friendly competition.
How to implement: Add a `Badge` Prisma model with `userId`, `badgeType`, `earnedAt`. Create a badge-evaluation service that runs after each assessment submission. Display badges in `ProfilePage.tsx` and `LearnerDashboard.tsx`. Add a streak counter to `User` model.

6

Medium Priority

Multilingual Expansion (Tamil, Telugu, Bengali)

What: Extend bilingual support beyond English/Hindi to include 3 additional Indian languages. NEP 2020 mandates mother-tongue instruction access.
How to implement: Extend `src/constants/translations.ts` with new language keys. Add language selector options in `AdminSettingsPage.tsx` and `Header.tsx`. Pass language to Gemini prompts in `AIPrompts` — already structured to accept language parameter.

7

Medium Priority

SWAYAM / NPTEL Live Course Sync (Web Scraping / API)

What: Automatically keep the course catalog fresh by fetching live course data from SWAYAM and NPTEL APIs instead of relying on manually seeded entries.
How to implement: Create a cron job (node-cron) in `server.ts` that runs daily: scrape or call SWAYAM's open course list, upsert into Prisma `Course` table. The frontend already fetches from `/api/courses` dynamically.

8

Medium Priority

Mobile App (React Native / Expo)

What: A native iOS & Android app sharing the same backend API. Government employees in field locations need offline assessment capability and push notifications.
How to implement: Bootstrap with `expo init StatSkillAIMobile`. Reuse all

`src/services/` TypeScript logic and `src/types/`. Replace Tailwind with NativeWind. Add Firebase Cloud Messaging for push notifications as a supplement to Socket.io.

9

Medium Priority

Advanced Analytics — Cohort Analysis & Predictive Scoring

What: Predict a learner's likely domain score after N weeks of following their learning path (ML regression model). Show cohort trends: "Learners with your profile improved Statistical scores by avg. 18 pts in 6 weeks."
How to implement: Export historical assessment data to a Python microservice (FastAPI). Train a simple linear regression model on domain×attempt×time features. Expose a `POST /api/analytics/predict` endpoint. Render prediction chart in `LearnerDashboard.tsx`.

🌟 Long-Term Vision Features

A

Long-Term

UPI-Based Premium Mentorship Marketplace

Enable verified mentors to offer paid consultation slots. Integrate Razorpay (UPI/NetBanking) for payment processing. StatSkillAI takes a platform fee. Monetisation enables platform sustainability.

B

Long-Term

National Credential Blockchain Registry

Issue competency certificates as NFT-like verifiable credentials on a public blockchain (Hyperledger Fabric / Polygon). Employers can verify authenticity via a public URL without a central authority.

C

Long-Term

DigiLocker Integration

Allow learners to pull educational certificates (10th, 12th, graduation) directly from their DigiLocker via UIDAI OAuth API. Pre-fill profile completeness fields automatically and attach as verified credentials to applications.

D

Long-Term

Adaptive Learning AI Tutor (Voice)

Add a voice-based AI tutor using Google Speech-to-Text + Text-to-Speech APIs. Learners can practice answering assessment questions verbally. AI evaluates response quality and provides spoken feedback — critical for rural & low-literacy users.

SECTION 09

Codebase File Structure

```
statskillai/
├── server.ts               # Main Express + Vite server entry point
├── package.json            # Dependencies & npm scripts
├── tsconfig.json           # TypeScript compiler configuration
├── vite.config.ts          # Vite bundler configuration (React + Tailwind)
├── index.html              # SPA root HTML file
├── .env                    # Local environment variables (not committed)
├── .env.example            # Template for required environment variables
│
├── prisma/
│   ├── schema.prisma       # Full PostgreSQL schema (16 models)
│   └── seed.ts             # Idempotent database seeder with demo data
│
├── server/
│   ├── db.ts               # Prisma singleton + health check function
│   ├── socket.ts           # Socket.io init, user rooms, message & notify helpers
│   ├── middleware/
│   │   └── auth.ts         # JWT, RBAC, password validation middleware
│   └── routes/
│       ├── auth.ts         # Register, login, me, refresh, logout, demo-switch
│       ├── profile.ts      # Profile CRUD + domain-scores aggregation
│       ├── assessments.ts  # Attempts history + submit + recalculate gap
│       ├── gaps.ts         # Gap analysis get + save (upsert)
│       ├── paths.ts        # Learning path CRUD + step status update
│       ├── courses.ts      # Course catalog CRUD
│       ├── opportunities.ts# Opportunities + applications + status pipeline
│       ├── mentorship.ts   # Mentors dir, mentorships, directives, DMs
│       ├── analytics.ts    # Workforce analytics aggregation (real + seeded)
│       └── quizzes.ts      # Quiz history get + save
│
└── src/
    ├── main.tsx            # React app bootstrap (createRoot)
    ├── App.tsx             # Routes, ProtectedRoute, AppLayout, notification toast
    ├── index.css           # Tailwind imports + accessibility overrides
    │
    ├── types/
    │   └── index.ts        # All TypeScript interfaces (16 domain models)
    │
    ├── constants/
    │   ├── competency.ts   # Domain definitions, weights, scoring functions
    │   └── translations.ts # Full bilingual EN/HI translation dictionary
    │
    ├── context/
    │   └── AppContext.tsx  # Global state: auth, user, profile, scores, notifications
    │
    ├── services/
    │   ├── apiService.ts   # Typed REST API client + Socket.io connection manager
    │   ├── storageService.ts# LocalStorage cache layer (offline-first fallback)
    │   ├── geminiService.ts# All Gemini AI function calls (questions, gaps, paths, chat)
    │   ├── aiPrompts.ts    # Structured Gemini system instructions & prompt templates
    │   ├── aiFallbacks.ts  # 60+ offline MCQ bank + fallback gap/path generators
    │   └── seedData.ts     # Client-side seed data (users, courses, mentors, etc.)
    │
```

```
├── components/
│   ├── common/
│   │   ├── Header.tsx   ScoreBadge.tsx   Sidebar.tsx
│   │   ├── NoticeBanner.tsx   DataTable.tsx   ConfirmModal.tsx
│   ├── learner/
│   │   ├── RadarCompetencyChart.tsx     AssessmentHistoryChart.tsx
│   │   └── FloatingAssistant.tsx
│   └── admin/
│       ├── SkillGapHeatmap.tsx   EmployeeDrilldownModal.tsx
│
└── pages/
    ├── LoginPage.tsx   AboutPage.tsx
    ├── learner/
    │   ├── LearnerDashboard.tsx      AdaptiveAssessmentPage.tsx
    │   ├── GapAnalysisPage.tsx       LearningPathPage.tsx
    │   ├── QuizGeneratorPage.tsx     OpportunitiesPage.tsx
    │   ├── MentorshipPage.tsx        ProfilePage.tsx
    ├── mentor/
    │   ├── MentorDashboard.tsx       MenteeDetailPage.tsx
    └── admin/
        ├── AdminAnalyticsPage.tsx    AdminCatalogPage.tsx
        ├── AdminApplicantsPage.tsx   AdminMentorsPage.tsx
        └── AdminSettingsPage.tsx
```

Quick Reference — Technology Versions

`Node.js v25.9` `React 19.0.1` `TypeScript 7.0.2` `Vite 8.3` `Express 4.21`
`Socket.io 4.8.4` `Prisma 6.19.3` `@google/genai 2.4` `Tailwind CSS 4.3`
`Recharts 3.10` `bcryptjs 3.0` `jsonwebtoken 9.0` `Helmet 8.3`
`express-rate-limit 8.7` `lucide-react 0.546` `Motion 12.23` `tsx 4.23`
`react-router-dom 7.18`

StatSkillAI

AI-Powered Competency & Career Development Platform

Aligned with National Education Policy (NEP) 2020 & Mission Karmayogi · CU Hackathon 2026

Built with: React 19 · Express · PostgreSQL · Prisma · Google Gemini · Socket.io · Tailwind CSS